

F-strings replaced these mechanisms by placing variables directly within the string using curly braces {}, as follows:

```
greeting = f"Hello, {name}! You are {age} years old."
```

Performance and readability improvements: F-strings are not just more readable but are usually a lot faster than the alternatives because they evaluate an expression during runtime and insert it directly into the string format. Hence, f-strings clearly outdo `.format()` and `%`.

On the readability side, the code is cleaner and more comprehensible. One does not have to worry any more about the order of placeholder or format specifiers.

```
product_name = "Laptop"
price = 999.99
quantity = 3

# Using f-string
message = f"Product: {product_name}, Price: ${price:.2f},
Quantity: {quantity}"
print(message)
```

The output is:

```
Product: Laptop, Price: $999.99, Quantity: 3
```

In this case, the f-string inserts the values of `product_name`, `price`, and `quantity` directly into the string. It also rounds the price to two decimal places.

F-strings provide a more straightforward, more efficient means of formatting strings and are thus the method of choice for Python programmers today.

Pattern matching (since version 3.10)

An advanced feature added in the Python 3.10 version is pattern matching, which supports more expressiveness and readability in the code while dealing with complex data structures. It makes it easier for programmers to match data against a given pattern and select appropriate portions of it in a simple and easily readable declarative style.

What it is and why it matters: Pattern matching allows you to check for specific shapes or values in data structures such as tuples, lists, dictionaries, etc. This is somewhere analogous to switch/case statements used in other programming languages, although it is far more powerful in the case of Python. Instead of a series of if/elif conditions, pattern matching lets you write simple and more readable code while working with different data structures or conditions.

The main advantage is that it provides a way of associating data to specific patterns and extracting

the values seamlessly, without the excessive use of conditional statements. This means cleaner and more understandable code, especially for complex conditions or nested data structures.

Real-world applications: Parsing instructions, extraction of data: In real-life scenarios such as command parsing, user input processing, and data extraction from complex structures, pattern-matching proves its worth. Consider a command-line argument parser with different options or flags for each command. Pattern matching allows the programmer to condense multiple cases into a single handling entity and avoid cluttering the code with different checks.

Another application is retrieving possible values from structured data — for example, JSON or XML. This is also where pattern matching can help locate values or structures fast, so that the data handling from APIs or files can be easier.

```
def describe_value(value):
    match value:
        case int():
            return "It's an integer!"
        case str():
            return "It's a string!"
        case list():
            return "It's a list!"
        case _:
            return "Unknown type"
```

```
print(describe_value(42)) # Output: It's an integer!
print(describe_value("hello")) # Output: It's a string!
```

In this case:

- The match statement tests the type of the value.
- The case statements test for different patterns (`int()`, `str()`, `list()`), and return the respective result.
- The `_` (underscore) case is a default for any value that fails to match the given patterns.

Pattern matching makes conditional checks easier and provides for more readable, understandable management of different data types or forms, and thus is an asset to Python developers.

Asynchronous programming with *async* and *await*

Asynchronous programming allows you to write programs that efficiently execute non-blocking operations, keeping them responsive. To make async programming more readable and easier, Python has added the keywords *async* and *await*. With the use of these keywords, you are able to declare asynchronous functions that can concurrently perform tasks without blocking the execution in the main thread, making them suitable for I/O operations, web requests, or multiple-event processing.